Red Hat Linux Experiemnt

Practical No. 05

1 Aim

To launch and access an Amazon EC2 Linux instance using SSH key-based authentication, perform basic Linux system administration, explore the Linux filesystem and user accounts, install and configure the Nginx web server, and host a custom HTML webpage accessible through the public IPv4 address of the EC2 instance.

2 Description

Configured an AWS EC2 Linux instance, connected through SSH, explored Linux commands and user accounts, installed and started Nginx, and hosted a custom HTML webpage accessible through the instance's public IP address.

3 Topology Diagram

The following diagram illustrates the architecture and communication between the AWS resources used in this practical.

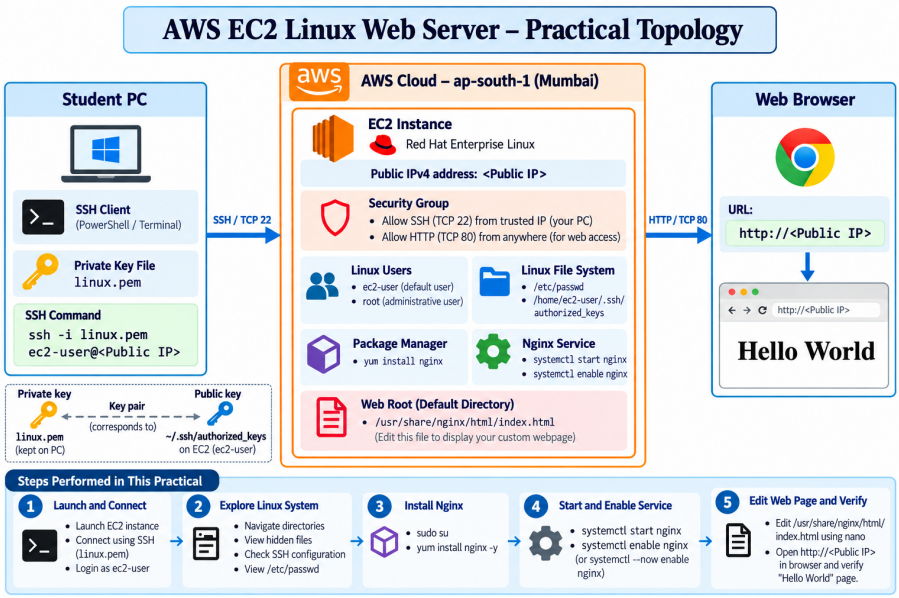


Figure 1: Topology Diagram

4 Procedure

4.1 Step 1: Create the linux instance by selecting the red hat linux

Description:

Create a instance with the Red Hat Enterprise linux

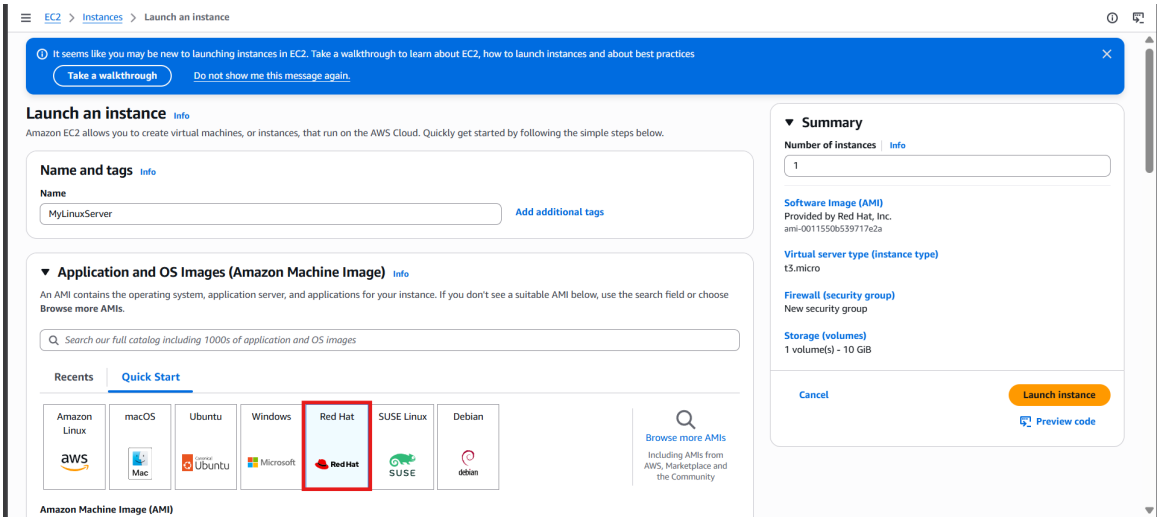


Figure 2: Step 1: First Step Title

4.2 Step 2: Configure the network settings for the instance

Description:

Like selecting the RDP for Windows instance we use SSH Protocol to establish our connection with our instance.

SSH (Secure Shell) is a network protocol used to securely connect to and manage a remote server. In AWS, SSH is commonly used to access Linux-based EC2 instances.

Why SSH Specifically?

- **Security:** Encrypts the connection between your computer and the server.
- **Command-line access:** Allows you to manage Linux systems using commands.
- **Key-based authentication:** AWS commonly uses a private key (`.pem` file) to authenticate your connection to the EC2 instance.
- **Remote access:** Allows you to manage your server without physically accessing the machine.

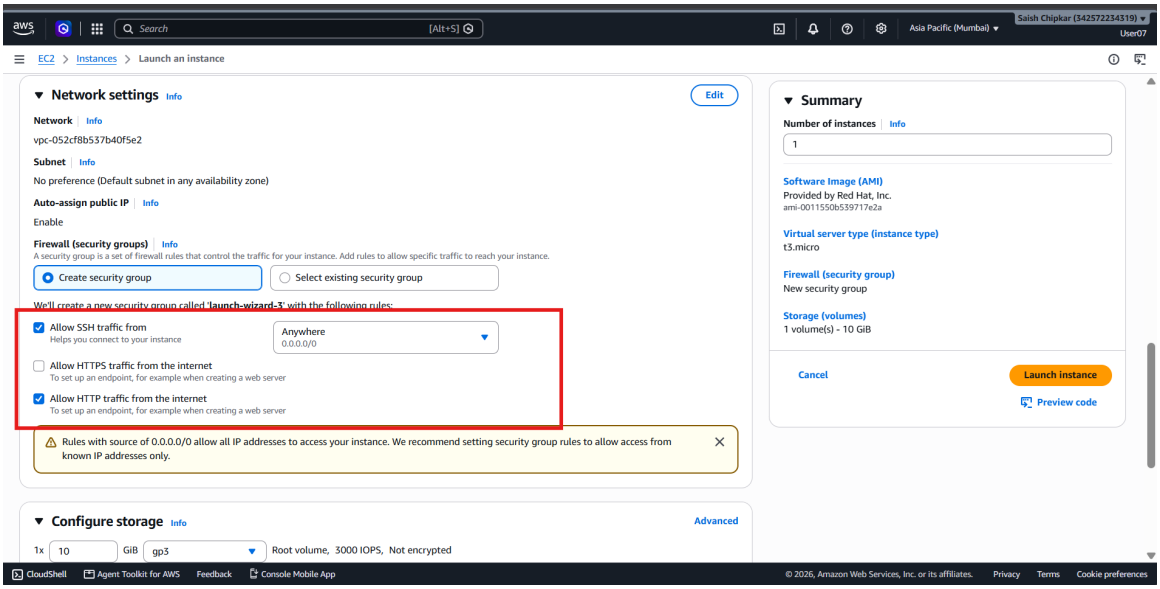


Figure 3: Step 1: First Step Title

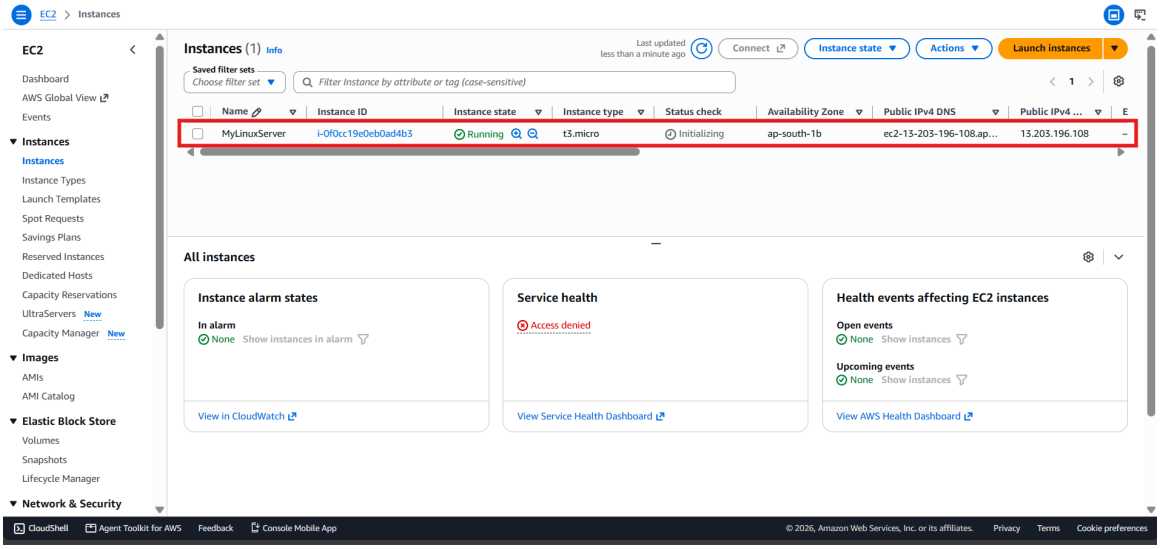


Figure 4: See the instance running

4.3 Step 3: Establish a SSH Connection from the console to the Instance

Description:

In this step, a secure connection was established between the local Windows computer and a Linux-based Amazon EC2 instance using the Secure Shell (SSH) protocol.

Instead of using the pre-generated SSH command provided by the AWS Management Console, the connection was established manually by specifying the public IPv4 address, Linux username, and private key file path.

The EC2 instance named `MyLinuxServer` was in the Running state. Its public IPv4 address

was `13.203.196.108`, and the default Linux username was `ec2-user`. The private key file `linux.pem` was stored in the Downloads directory on the local Windows computer.

SSH Command:

```
ssh -i "C:\Users\Saish Chipkar\Downloads\linux.pem" ec2-user@13.203.196.108
```

Explanation of the Command:

- `ssh`**:** Initiates a secure remote connection to the Linux server.
- `-i`**:** Specifies the identity file used for authentication.
- **Private key path:** Specifies the location of the `linux.pem` private key on the local Windows computer. Quotation marks are necessary because the path contains spaces.
- `ec2-user`**:** Specifies the Linux user account used to access the instance.
- **Public IPv4 address:** Identifies the remote EC2 instance at `13.203.196.108`.

Connection Verification:

During the first connection, SSH displayed a host authenticity warning because the server's host key was not yet stored locally. After the intended server was verified, `yes` was entered to trust the host key and add it to the local known-hosts file.

After successful authentication, the Linux shell prompt `[ec2-user@ip-172-31-6-154 ~]$` appeared, confirming that the remote SSH session was established successfully.

Why SSH Was Used:

- **Security:** Encrypts communication between the local computer and the server.
- **Key-based authentication:** Uses the private key to authenticate access to the instance.
- **Remote access:** Allows administration without physically accessing the server.
- **Command-line management:** Enables execution of Linux commands to manage files, users, permissions, and services.

Note: SSH access requires network connectivity and an inbound security group rule allowing TCP port 22 from the connecting computer's IP address. The private key must be kept confidential.

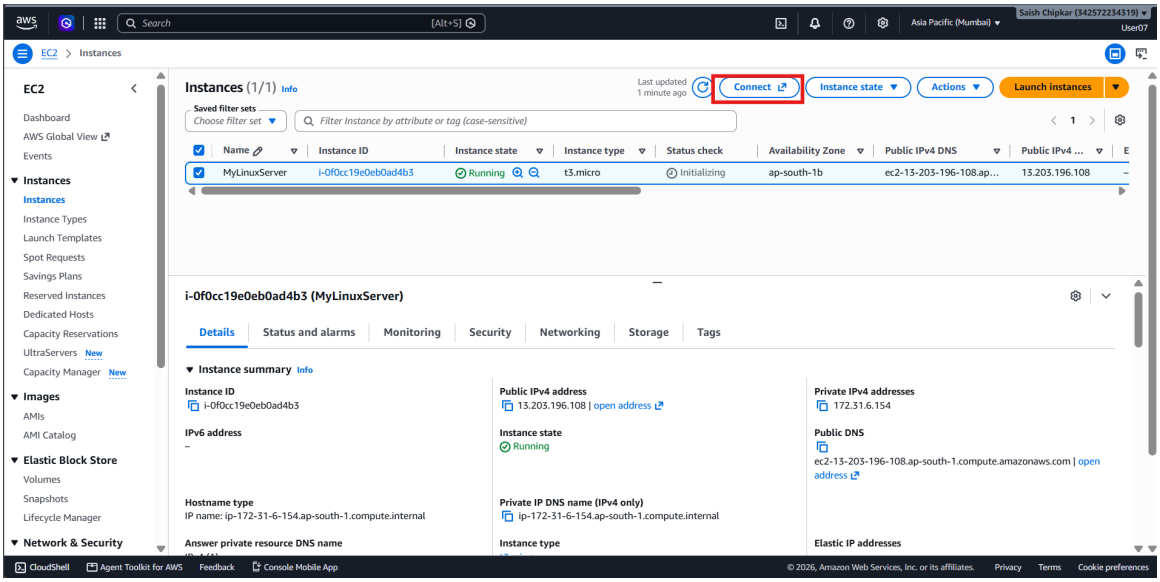


Figure 5: Click on the connect for the instance

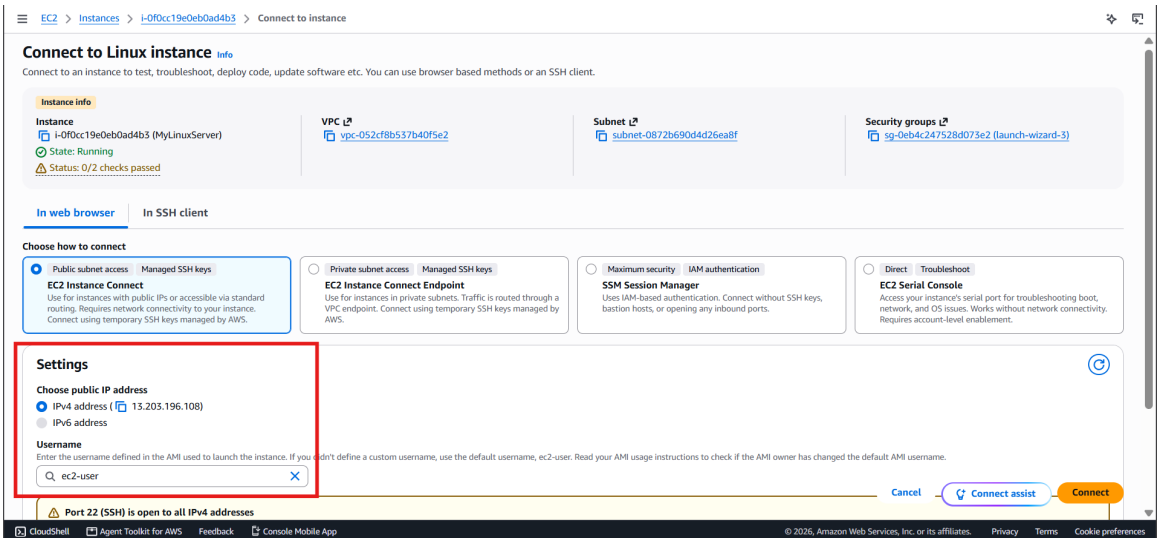


Figure 6: Copy the public IPV4 Address

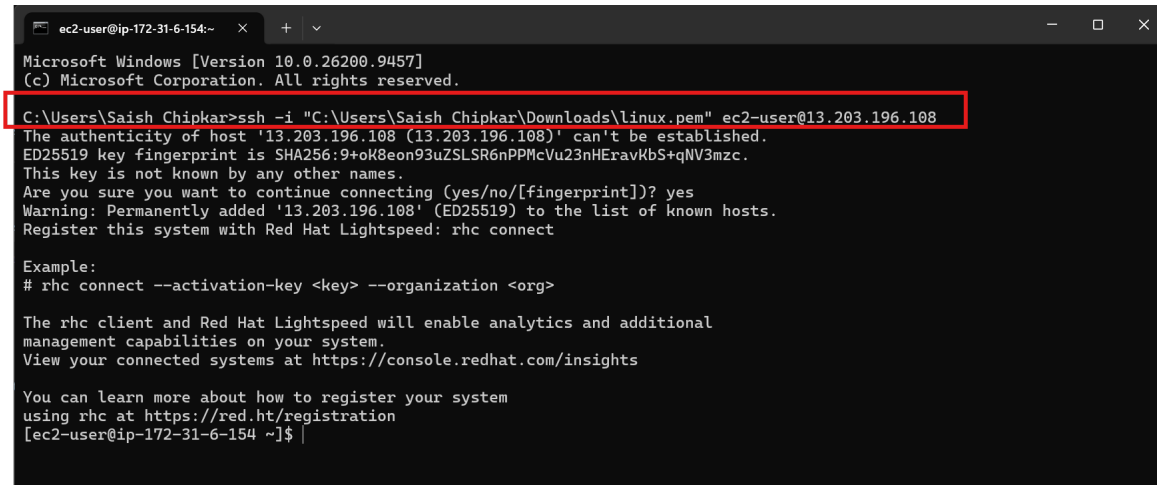


Figure 7: Run the command

4.4 Step 4: Exploring SSH Configuration and Authorized Keys

Description:

In this step, the SSH configuration directory of the `ec2-user` account was explored to understand how public-key authentication works in an AWS EC2 Linux instance. Various Linux commands were executed to navigate the filesystem and inspect the SSH-related files.

1. Checking the Current Directory

The `pwd` command was executed to display the current working directory.

```
/home/ec2-user
```

This confirms that the terminal was operating inside the home directory of the `ec2-user` account.

2. Listing Files and Hidden Files

The `ls` command lists visible files and directories, whereas `ls -a` displays all entries, including hidden files whose names begin with a dot.

The output included files such as `.bash_logout`, `.bash_profile`, `.bashrc`, and the hidden directory `.ssh`.

3. Accessing the SSH Directory

The following command was used to enter the SSH directory:

```
cd .ssh
```

After entering the directory, the `ls` command displayed the file `authorized_keys`.

The file is located at:

```
/home/ec2-user/.ssh/authorized_keys
```

4. Examining the Authorized Keys File

The following command was executed to display the contents of the file:

```
cat authorized_keys
```

The output contained an SSH public-key entry beginning with `ssh-rsa`, followed by encoded public-key data and a comment identifying the key.

The `authorized_keys` file stores public keys that are permitted to authenticate as the associated Linux user. The comment at the end of a key entry is only a descriptive label and does not grant permissions.

5. Relationship Between the Private and Public Keys

During the earlier SSH connection, the Windows computer used the private key file `linux.pem` to connect to the EC2 instance.

The private key remained on the local computer, while the corresponding public key was authorized on the server. During authentication, the SSH client proves possession of the private-key credential, and the server checks whether the corresponding public key is authorized.

When authentication succeeds, the user gains access to the Linux shell, subject to the applicable access controls.

Purpose of This Step:

- **Filesystem navigation:** Learned to use commands such as `pwd`, `ls`, `ls -a`, and `cd`.
- **Hidden files:** Understood that filenames beginning with a dot are hidden by default.
- **SSH configuration:** Identified the `.ssh` directory and its `authorized_keys` file.
- **Key-based authentication:** Understood how the local private key corresponds to a public key authorized on the remote server.

Security Note:

The `authorized_keys` file contains public-key entries, not private keys. The private key file `linux.pem` must be kept confidential and should never be shared publicly.

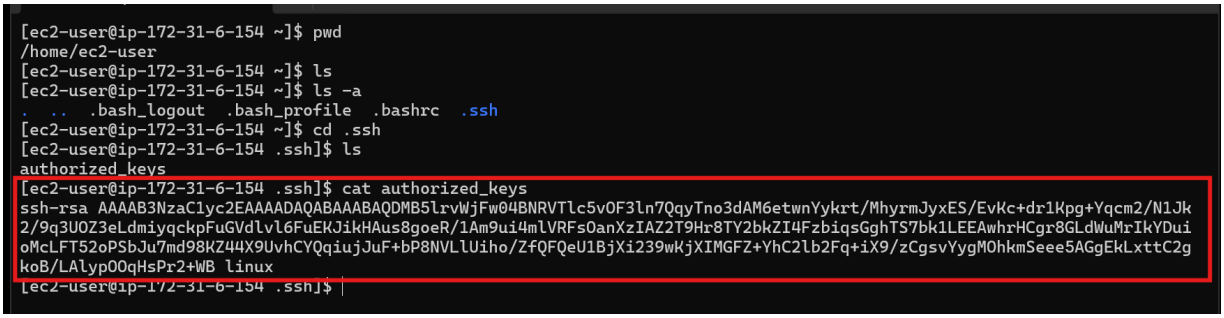


Figure 8: Exploring the SSH Directory and Authorized Keys

4.5 Step 5: Exploring the Linux Root Directory and User Accounts

Description:

In this step, the Linux root directory was explored, and the `/etc/passwd` file was examined to understand the Linux filesystem structure and user account management.

1. Listing the Root Directory

The `ls` command was executed from the root directory to display its contents.

```
ls
```

The output displayed directories such as `bin`, `boot`, `dev`, `etc`, `home`, `lib`, `mnt`, `opt`, `root`, `tmp`, `usr`, and `var`.

These directories serve different purposes in Linux. For example, `/etc` stores system configuration files, `/home` contains regular users' home directories, and `/var` stores variable data such as logs.



Figure 9: Listing the Linux Root Directory

2. Navigating to the Root Directory

The following command was executed:

```
cd ../..
```

This command moves two directory levels upward. Since the terminal was initially in the user's home directory, the command brought the session to the root directory, represented by `/`.

3. Viewing the Linux User Account File

The following command was executed to display the contents of the user account database:

```
cat /etc/passwd
```

The `/etc/passwd` file contains one entry for each local user account, including system accounts used by services and regular interactive users.

Each line contains seven colon-separated fields:

```
username:password:UID:GID:GECOS:home:shell
```

- **Username:** The name of the account, such as `root` or `ec2-user`.
- **Password field:** Usually contains `x`, indicating that password information is stored separately in a protected file.
- **UID:** The numeric user ID that identifies the account.
- **GID:** The numeric ID of the user's primary group.
- **GECOS:** An optional description or comment associated with the account.
- **Home directory:** The directory associated with the account.
- **Login shell:** Specifies the shell or program launched for the account.

4. Understanding Different Linux Accounts

The output contained several types of accounts.

- **Root account:** The superuser account with UID 0, which has administrative privileges.
- **Regular user:** The `ec2-user` account has UID 1000 and home directory `/home/ec2-user`. Its shell is `/bin/bash`.
- **System accounts:** Accounts such as `sshd`, `chrony`, and `dbus` support system services and processes.
- **Non-login accounts:** Many service accounts use `/sbin/nologin` as their configured shell to prevent ordinary interactive logins through that shell.

These accounts allow Linux to separate service identities and privileges instead of running every service as the root user.

```
[ec2-user@ip-172-31-6-154 ~]$ cd ../..
[ec2-user@ip-172-31-6-154 /]$ cat /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/nologin
daemon:x:2:2:daemon:/sbin:/usr/sbin/nologin
adm:x:3:4:adm:/var/adm:/usr/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/usr/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/usr/sbin/nologin
operator:x:11:0:operator:/root:/usr/sbin/nologin
games:x:12:100:games:/usr/games:/usr/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/usr/sbin/nologin
nobody:x:65534:65534:Kernel Overflow User:/:/usr/sbin/nologin
yggdrasil:x:999:999:yggdrasil system user:/var/lib/yggdrasil:/usr/sbin/nologin
yggdrasil-worker:x:998:998:yggdrasil worker user:/:/usr/sbin/nologin
tss:x:59:59:Account used for TPM access:/:/usr/sbin/nologin
systemd-oom:x:997:996:systemd Userspace OOM Killer:/:/sbin/nologin
dbus:x:81:81:System Message Bus:/:/usr/sbin/nologin
polkitd:x:114:114:User for polkitd:/:/sbin/nologin
dhcpcd:x:996:995:Minimalistic DHCP client:/var/lib/dhcpcd:/usr/sbin/nologin
sssd:x:995:994:User for sssd:/run/sssd/:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/usr/share/empty.sshd:/usr/sbin/nologin
chrony:x:994:993:chrony system user:/var/lib/chrony:/sbin/nologin
systemd-coredump:x:992:992:systemd Core Dumper:/:/usr/sbin/nologin
ec2-user:x:1000:1000:Cloud User:/home/ec2-user:/bin/bash
[ec2-user@ip-172-31-6-154 /]$
```

Figure 10: Viewing Linux User Accounts in /etc/passwd

Result:

The Linux root directory structure and the `/etc/passwd` file were successfully explored. The purpose of important system directories, the seven fields of a user account entry, and the differences between regular and service accounts were understood.

```
[ec2-user@ip-172-31-6-154 /]$ ls
afs  bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
```

Figure 11: Exploring the root directory

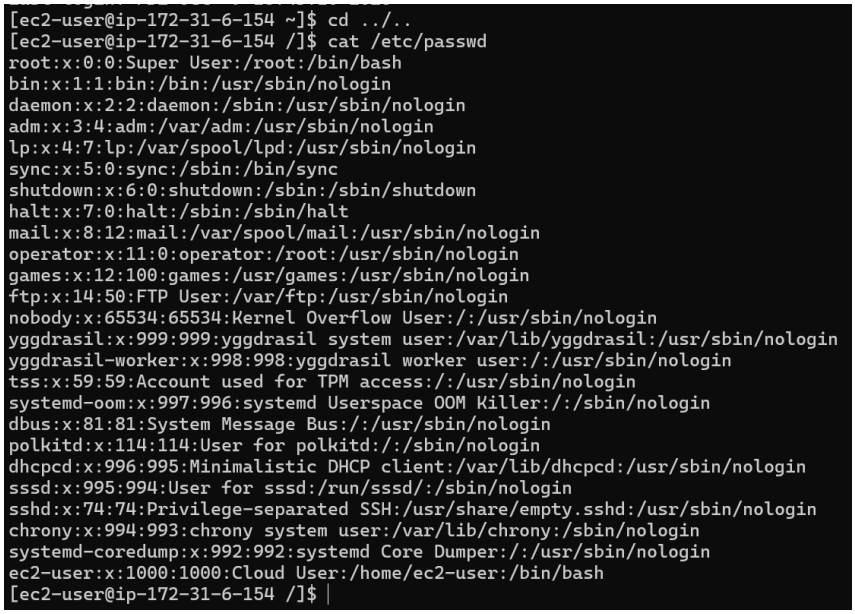


Figure 12: Exploring the various human and service accounts

4.6 Step 6: Installing and Configuring the Nginx Web Server

Description:

In this step, the Nginx web server was installed and configured on the AWS EC2 instance running Red Hat Enterprise Linux. The purpose was to host a web page on the virtual machine and verify that it could be accessed remotely through a web browser.

1. Switching to the Root User

The following command was executed to obtain administrative privileges:

```
sudo su
```

This command switches the current shell to the root user, allowing administrative tasks such as installing software and managing system services.

2. Installing Nginx

The following command was used to install Nginx:

```
yum install nginx -y
```

The `yum` package manager resolved the dependencies, downloaded the required packages, and installed Nginx along with its supporting components.

The `-y` option automatically answers yes to confirmation prompts during installation.

The terminal displayed a subscription-management message indicating that the system was not registered with an entitlement server. However, the package installation proceeded using the available repositories.

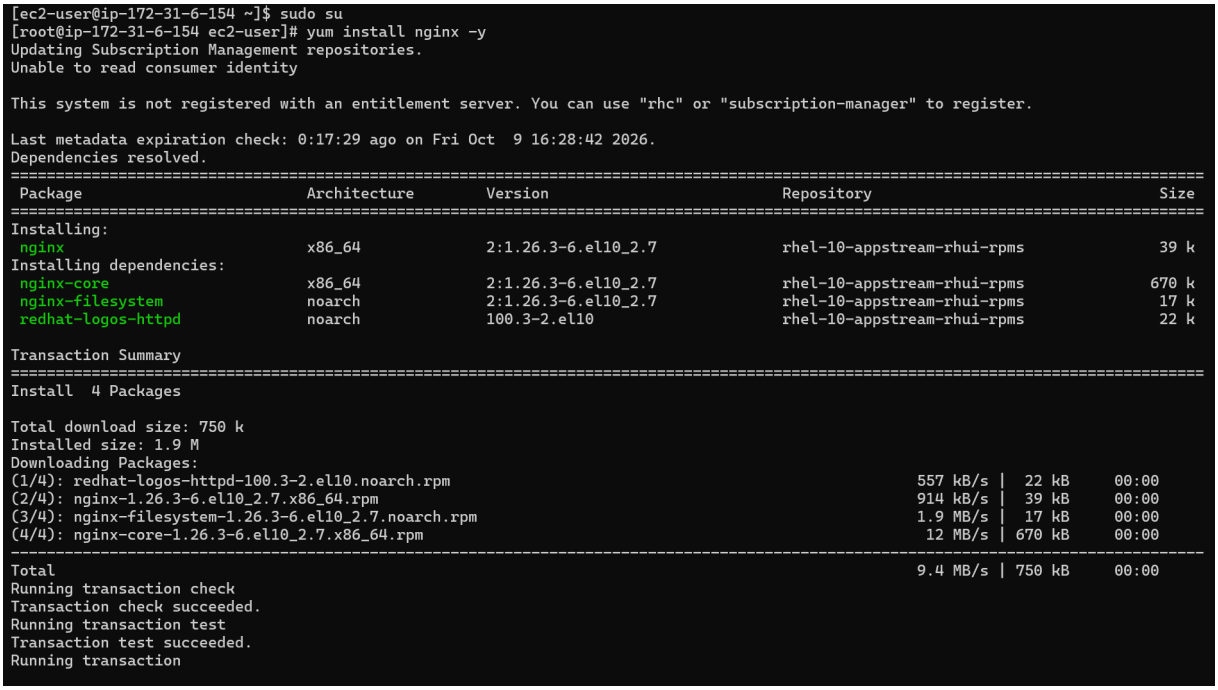


Figure 13: Installing Nginx Using the YUM Package Manager

3. Starting the Nginx Service

After installation, the following command was executed:

```
systemctl start nginx
```

The `systemctl` utility is used to manage services on Linux systems. The `start` action starts the Nginx service immediately.

4. Enabling Nginx at System Startup

The following command was executed:

```
systemctl enable nginx
```

This command configures Nginx to start automatically during subsequent system boots. Enabling a service does not, by itself, start it immediately; that was handled by the preceding command.

The `systemctl --now enable nginx` command enables Nginx to start automatically at boot and starts the service immediately, combining the functions of `systemctl enable nginx` and `systemctl start nginx`.

5. Verifying the Nginx Service Status

The following command was used to check the service:

```
systemctl status nginx
```

The output showed the service as `active (running)` and `enabled`. This confirmed that Nginx was running and configured to start automatically when the instance boots.

```
[root@ip-172-31-6-154 ec2-user]# systemctl start nginx
[root@ip-172-31-6-154 ec2-user]# systemctl enable nginx
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/usr/lib/systemd/system/nginx.service'.
[root@ip-172-31-6-154 ec2-user]# systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
     Active: active (running) since Fri 2026-10-09 16:46:55 UTC; 23s ago
 Invocation: 452501916f594f1f9793b721bd268429
   Main PID: 2120 (nginx)
      Tasks: 3 (limit: 4096)
     Memory: 3.1M (peak: 3.2M)
        CPU: 44ms
     CGroup: /system.slice/nginx.service
             ├─2120 "nginx: master process /usr/sbin/nginx"
             ├─2121 "nginx: worker process"
             └─2122 "nginx: worker process"

Oct 09 16:46:55 ip-172-31-6-154.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy se>
Oct 09 16:46:55 ip-172-31-6-154.ap-south-1.compute.internal nginx[2117]: nginx: the configuration file /etc/nginx/nginx.conf syntax >
Oct 09 16:46:55 ip-172-31-6-154.ap-south-1.compute.internal nginx[2117]: nginx: configuration file /etc/nginx/nginx.conf test is suc>
Oct 09 16:46:55 ip-172-31-6-154.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy ser>
lines 1-17/17 (END)
```

Figure 14: Nginx Service Running and Enabled

6. Testing the Web Server in a Browser

To verify remote access, the public IPv4 address of the EC2 instance was entered into a web browser using HTTP:

`http://<EC2-Public-IPv4-Address>`

The Red Hat Enterprise Linux test page appeared, indicating that an HTTP server was responding at the specified address. This confirmed that the web server was reachable from the browser.

The displayed default test page also indicated that the default web content had not yet been replaced with a custom webpage.

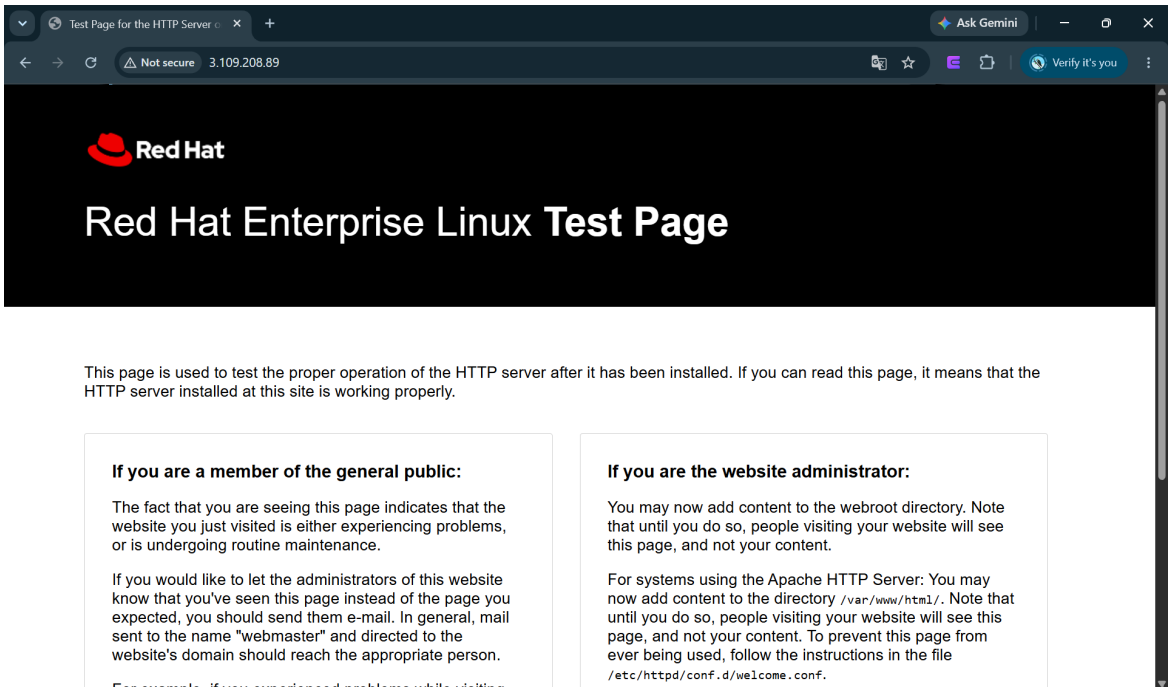


Figure 15: Verifying the Web Server Through a Web Browser

Note:

For browser access to work, the EC2 security group must allow inbound HTTP traffic on TCP port 80. The instance must also have suitable network connectivity and firewall settings. The browser screenshot confirms that an HTTP server was reachable, although the displayed Red Hat test page alone does not conclusively establish that Nginx, rather than another HTTP server, served the response.

Result:

Nginx was installed on the EC2 instance, started successfully, and enabled to start automatically during system boot. The service status was verified, and the web server was tested through a browser using the instance's public IPv4 address.

4.7 Step 7: Editing the Default HTML Page

Description:

In this step, the default web page served by the web server was replaced with a custom HTML page. The purpose was to understand how website content is stored on the EC2 instance and how changes to HTML files are reflected in a browser.

1. Navigating to the Web Root Directory

After installing and starting the web server, the terminal was used to navigate to the directory containing the website files.

For Nginx, the default document root on this system is typically:

```
/usr/share/nginx/html
```

The directory was accessed using the `cd` command. Administrative privileges were used to modify the website files.

2. Opening the HTML File Using Nano

The following command was executed to open the HTML file for editing:

```
nano index.html
```

The Nano text editor was used to create or modify the contents of `index.html`. The following HTML heading was entered into the file:

```
<h1>Hello World</h1>
```

The `h1` element represents a top-level heading in HTML. When rendered by a browser, it displays the text "Hello World" as a large heading.

After editing, the file was saved and the Nano editor was closed.

```
[root@ip-172-31-6-154 html]# nano index.html
```

Figure 16: Opening the HTML File in the Nano Editor

3. Saving the HTML File

After entering the HTML content, the changes were saved to `index.html`. Since the file is located in the web server's document root, the web server can return the updated content when a browser requests the page.

No restart is normally required for Nginx after editing a static HTML file, because the file is read when the HTTP request is handled.

4. Accessing the Custom Web Page

The web browser was opened on the local computer, and the public IPv4 address of the EC2 instance was entered into the address bar:

```
http://<EC2-Public-IPv4-Address>
```

The browser displayed the heading "Hello World". This confirmed that the updated HTML page was being served and was accessible through the instance's public IP address.



Figure 17: Custom HTML Page Displayed in the Browser

5. Understanding the Web Hosting Process

The process demonstrates how a static web page is hosted on a cloud-based virtual machine:

- **HTML file:** Defines the content displayed by the browser.

- **Document root:** Stores the web files served by the web server.
- **Nginx:** Receives HTTP requests and returns the requested web content.
- **Public IPv4 address:** Allows clients to reach the EC2 instance over the network when the required network rules permit it.
- **Web browser:** Sends an HTTP request and renders the HTML response.



Figure 18: Verification of the Hosted Web Page

Result:

The HTML file was edited successfully, and the custom "Hello World" heading was displayed in the web browser using the EC2 instance's public IPv4 address. This demonstrated the basic process of hosting and updating a static webpage on an AWS EC2 Linux instance.

5 Result

The AWS EC2 practical was completed successfully. A Linux-based EC2 instance was accessed remotely from a Windows computer using SSH and key-based authentication with a private key file (`.pem`). Linux directory structures, hidden files, SSH configuration, and user accounts were explored using commands such as `pwd`, `ls`, `cd`, and `cat`.

The Nginx web server was installed using the YUM package manager, started, and enabled to run automatically during system startup. The HTML file `index.html` was edited using the Nano text editor, and a custom "Hello World" webpage was successfully displayed in a web browser using the EC2 instance's public IPv4 address.

6 Conclusion

This practical provided hands-on experience with AWS EC2 instance management, remote Linux administration, SSH authentication, and web server configuration. It helped develop an understanding of Linux filesystem navigation, user account management, and the role of public and private keys in secure remote access.

Furthermore, installing and configuring Nginx demonstrated how a cloud-based Linux server can host static web content and make it accessible over HTTP. Editing the HTML file and verifying the result in a browser illustrated the complete process of deploying and testing a basic webpage on an AWS EC2 instance.

Overall, the practical strengthened the understanding of cloud computing, Linux command-line operations, and basic web hosting using AWS infrastructure.